The Hive: Pipeline and System Architecture Report

Deep Technical Architecture, DAG Modeling, and Execution Data Flow

---

This report provides an architectural deep-dive into The Hive's system pipeline. It details the internal mechanics of the Python orchestration engine, the exact data structures passed between agents, and the Directed Acyclic Graph (DAG) state management that makes the autonomous execution possible.

I. System Architecture Overview

The system architecture is decoupled into three primary layers: the Ingestion Layer, the Orchestration Layer (the Pipeline), and the Output Layer. The Orchestration Layer relies entirely on an offline LLM engine (Ollama running Llama 3) to process natural language without internet dependencies.

Zero Cloud Footprint
Because the architecture relies on local Ollama integration, all data traversing the pipeline remains in local memory or local disk. No external API keys are required for core execution.

II. The Pipeline Engine (src/pipeline.py)

The `pipeline.py` file contains the `SimulationPipeline` class, which is responsible for state machine transitions. It enforces a strict Directed Acyclic Graph (DAG) execution model. An agent cannot execute until its prerequisite agent has successfully returned a structured JSON state object.

A. State Management

The state object is a Python dictionary that grows as it moves through the pipeline. It begins containing only the raw threat report and ends containing hundreds of execution logs and detection rules. At every transition, `pipeline.py` writes the state to `intermediate_{step}.json`.

```
{
  "intel": { "techniques": ["T1059.001", "T1027"] },
  "plan": { "phases": {
      "access": ["T1059.001"],
      "evasion": ["T1027"]
  }},
  "execution_results": {
      "access": { "logs": [...], "success": true }
  },
  "purple_team": {
      "sigma_rules": [...],
      "gaps": [...]
  }
}
```

III. Execution Data Flow

1. Ingestion & Routing

The pipeline initializes by loading `config/agents.yaml`. This file defines the system prompts, temperatures, and constraints for the LLMs. The `run.py` script then passes the target JSON report into the `SimulationPipeline.run()` method.

2. Planning Constraints

When the Planner Agent receives techniques from the Intel Agent, it runs a dependency validation algorithm. It maps the techniques to 5 hardcoded phases: Reconnaissance, Initial Access, Persistence, Lateral Movement, and Exfiltration. If a technique is unsupported, it drops it and logs a warning.

3. Dynamic Instantiation of Operators

The pipeline loops through the 5 phases dynamically. For each phase that contains techniques, it dynamically instantiates the correct Operator class (e.g., `ReconAgent`, `AccessAgent`). This prevents memory bloat, as agents are only loaded into RAM when their specific phase is active.

IV. Inter-Component Interaction Protocol

The Hive avoids unstructured chatter between agents. Instead, it uses a highly structured payload protocol.

1. Request Framing: The pipeline wraps the previous phase's JSON output in a strict prompt template before querying the LLM.
2. JSON Enforcement: The LLM is instructed to respond ONLY in valid JSON. The `pipeline.py` engine includes an auto-retry mechanism. If the LLM hallucinates markdown or conversational text, the pipeline strips it or issues a

regeneration request.
3. Live Integration (Future): The Operator agents contain scaffolding to hook into Atomic Red Team's `atomic-operator` Python library. In live mode, the JSON structured techniques are passed directly to the `atomic-operator` execution engine, bridging AI reasoning with deterministic script execution.

V. Fault Tolerance and Error Handling

> **Pipeline Resilience**
> If the LLM fails to generate valid Sigma rules during the Purple Team phase, the pipeline does not crash. It falls back to generating a generic detection scorecard and logs the error in the final report.md.

The pipeline implements strict exception boundaries around agent invocations. If the Access Agent fails to simulate an exploit due to LLM context limits, the pipeline logs a 'Phase Failure', preserves the state, and proceeds to the Persistence phase if applicable. This mimics real-world red teaming where one failed exploit does not necessarily stop the entire campaign.